

Лабораторная работа №4

cm_timer.py

```
import time

from contextlib import contextmanager


class cm_timer_1:

    def __enter__(self):
        self.start = time.time()

    def __exit__(self, exc_type, exc_val, exc_tb):
        print(f'time: {time.time() - self.start}')


@contextmanager
def cm_timer_2():
    start = time.time()
    yield
    print(f'time: {time.time() - start}')


if __name__ == '__main__':
    from time import sleep
    with cm_timer_1():
        sleep(1.5)
    with cm_timer_2():
        sleep(1.5)
```

field.py

```
def field(items, *args):
    assert len(args) > 0
    for item in items:
        if len(args) == 1:
            value = item.get(args[0])
            if value is not None:
                yield value
        else:
            result = {arg: item.get(arg) for arg in args if item.get(arg) is not None}
            if result:
                yield result
```

```

if __name__ == '__main__':
    goods = [
        {'title': 'Ковер', 'price': 2000, 'color': 'green'},
        {'title': 'Диван для отдыха', 'color': 'black'}
    ]
    for i in field(goods, 'title'):
        print(i)
    for i in field(goods, 'title', 'price'):
        print(i)

```

gen_random.py

```

import random

def gen_random(num_count, begin, end):
    for _ in range(num_count):
        yield random.randint(begin, end)

```

```

if __name__ == '__main__':
    for i in gen_random(5, 1, 3):
        print(i)

```

print_result.py

```

def print_result(func):
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        print(func.__name__)
        if isinstance(result, dict):
            for k, v in result.items():
                print(f'{k} = {v}')
        elif isinstance(result, list):
            for i in result:
                print(i)
        else:
            print(result)
        return result
    return wrapper

```

```

@print_result
def test_1():
    return 1

```

```
@print_result
```

```
def test_2():
```

```
    return 'iu5'
```

```
@print_result
```

```
def test_3():
```

```
    return {'a': 1, 'b': 2}
```

```
@print_result
```

```
def test_4():
```

```
    return [1, 2]
```

```
if __name__ == '__main__':
```

```
    print('!!!!!!!')
```

```
    test_1()
```

```
    test_2()
```

```
    test_3()
```

```
    test_4()
```

process_data.py

```
import json
```

```
import sys
```

```
from gen_random import gen_random
```

```
from unique import Unique
```

```
from print_result import print_result
```

```
from cm_timer import cm_timer_1
```

```
path = sys.argv[1]
```

```
with open(path, encoding='utf-8') as f:
```

```
    data = json.load(f)
```

```
@print_result
```

```
def f1(arg):
```

```
    return sorted(Unique([i['job-name'] for i in arg], ignore_case=True), key=str.lower)
```

```

@print_result
def f2(arg):
    return list(filter(lambda x: x.lower().startswith('программист'), arg))

```

```

@print_result
def f3(arg):
    return list(map(lambda x: f'{x} с опытом Python', arg))

```

```

@print_result
def f4(arg):
    salaries = gen_random(len(arg), 100000, 200000)
    return [f'{x}, зарплата {s} руб.' for x, s in zip(arg, salaries)]

```

```

if __name__ == '__main__':
    with cm_timer_1():
        f4(f3(f2(f1(data))))

```

sort.py

```

def abs_key(x):
    return abs(x)

```

```

data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]

```

```

if __name__ == '__main__':
    result = sorted(data, key=abs_key, reverse=True)
    print(result)

```

```

result_with_lambda = sorted(data, key=lambda x: abs(x), reverse=True)
print(result_with_lambda)

```

unique.py

```

class Unique:
    def __init__(self, items, **kwargs):
        self.items = iter(items)
        self.ignore_case = kwargs.get('ignore_case', False)
        self.seen = set()

```

```
def __iter__(self):
```

```
    return self
```

```
def __next__(self):
```

```
    while True:
```

```
        item = next(self.items)
```

```
        key = item.lower() if self.ignore_case and isinstance(item, str) else item
```

```
        if key not in self.seen:
```

```
            self.seen.add(key)
```

```
        return item
```

```
if __name__ == '__main__':
```

```
    data = [1, 1, 2, 2, 3]
```

```
    for i in Unique(data):
```

```
        print(i)
```

```
from gen_random import gen_random
```

```
for i in Unique(gen_random(10, 1, 3)):
```

```
    print(i)
```

```
data = ['a', 'A', 'b', 'B', 'a']
```

```
for i in Unique(data):
```

```
    print(i)
```

```
for i in Unique(data, ignore_case=True):
```

```
    print(i)
```

Работа программы

```
(venv) deviatovilya@MacBook-Air-cua-Deviatov Labs-2025-python % python ./process_data.py ./data_light.json
f1
Программист C#
Программист C++
Программист Java
Программист Python
Тестировщик
f2
Программист C#
Программист C++
Программист Java
Программист Python
f3
Программист C# с опытом Python
Программист C++ с опытом Python
Программист Java с опытом Python
Программист Python с опытом Python
f4
Программист C# с опытом Python, зарплата 148718 руб.
Программист C++ с опытом Python, зарплата 159825 руб.
Программист Java с опытом Python, зарплата 145347 руб.
Программист Python с опытом Python, зарплата 192648 руб.
time: 9.918212890625e-05
```